

Verifier-Grounded Evaluation of LLM-Generated Network Configurations: A Preregistered NetConfig-Semantic Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We report a fully preregistered, artifact-anchored paired experiment (CAND-2026-001-NetConfig-Semantic-v1; preregistration SHA-256 7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fdebb71a37) comparing a deterministic template baseline to a single-pass LLM generator (declared_model_version = gpt-5-mini) on N = 120 synthetic intent→configuration tasks using a deterministic validator. Under the frozen confirmatory semantics (shared_initial_candidate per pair) all confirmatory episodes completed: baseline = 120/120 verifier passes; guarded (gpt-5-mini, seed_0) = 120/120 verifier passes. Paired difference = 0.00; 95% bootstrap percentile CI [0.00, 0.00] (10,000 resamples, seed = 1729); exact McNemar two-sided p = 1.0. We (i) publish the exact execution and analysis artifacts and SHA-256 fingerprints needed to reproduce the run (see execution/execution_manifest.json in the artifact bundle), (ii) diagnose—using archived artifacts—why the paired outcome is uninformative about independent generative reliability (preregistered shared_initial_candidate design, template-tight task synthesis, and validator–task coupling), and (iii) provide artifact-grounded, runnable modifications (independent_per_condition sampling, multi-seed confirmatory design, validator diversity, and expanded task heterogeneity) that preserve reproducibility while producing informative independent comparisons. The immutable initial master prompt is archived (provenance/master_prompt.txt; SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdff1582bb39d15ba1).

I. INTRODUCTION

Motivation. Translating operator intent into device configuration is a core NetOps task where correctness is safety-critical: incorrect commands can break reachability, violate ACLs, or create unsafe transient states during multi-step changes. Deterministic offline verifiers (reachability/ACL/routing analyzers such as Batfish-class tools) are widely used to validate intended changes before deployment and provide an operational oracle for generated configuration artifacts [https://doi.org/10.1145/3603269.3604866]. Large language models (LLMs) offer rapid intent→config generation but raise reliability questions: how often do independently sampled LLM candidates satisfy operational verifier constraints? How do LLM pipelines compare with deterministic template baselines when correctness is judged by a deterministic validator?

Research question and contribution. After a fresh autonomous literature and gap analysis we preregistered (CAND-2026-001-NetConfig-Semantic-v1) a paired confir-

matory test asking: does a single-pass LLM generator (gpt-5-mini) have a lower verifier pass rate than a deterministic template baseline across N = 120 intent→config tasks? Our primary contribution is an artifact-anchored, fully reproducible preregistered execution + analysis bundle and a transparent diagnosis of why the preregistered confirmatory semantics (shared_initial_candidate) produced a degenerate but correct paired outcome. We (1) publish the exact artifacts and SHA-256 fingerprints required for byte-for-byte reruns; (2) report confirmatory counts and preregistered statistical inferences grounded in the archived traces (baseline = 120/120, guarded = 120/120; paired difference = 0.00; bootstrap CI [0.00,0.00]; McNemar p = 1.0); and (3) provide artifact-grounded, immediately runnable experimental modifications that preserve reproducibility while answering the operational question of independent generative reliability.

II. RELATED WORK

Verifier-grounded NetOps and intent→config. Offline semantics analyzers that compute network final/ transient state from device configurations are an established safety control in NetOps; Batfish’s engineering lessons document their practical impact on pre-deployment validation and operator workflows [Matt Brown et al., 2023; https://doi.org/10.1145/3603269.3604866]. Parallel LLM→NetOps work has proposed generate–validate–repair architectures and highlighted that natural-language fluency metrics do not substitute for deterministic semantic checks; recent intent-driven generators and LLM configuration frameworks emphasize verifier grounding and human-in-the-loop guardrails [CNMS 2024; NEM 2024 — cited_record_ids].

Methodological gap this study addresses. Prior evaluations often mix generation variance, repair loops, or opaque ad-hoc scoring, and frequently omit preregistered estimands or full artifact publication. Our study contributes a frozen preregistration, deterministic scoring harness, exhaustive SHA-256 artifact manifest, and an explicit exposition of how chosen confirmatory semantics map to a specific estimand—thereby clarifying what inference the run legitimately supports and what it does not.

III. METHODOLOGY

Preregistration and experiment plan. We froze the study prior to observing model outputs: CAND-2026-001-NetConfig-Semantic-v1 (preregistration SHA-256 7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978eq76bb59fde9db71a37). Primary estimand: paired_success_rate_difference_guarded_minus_baseline computed on N = 120 paired tasks (40 tasks per topology class: edge, datacenter, multi-AS). Confirmatory generation semantics were preregistered as 'shared_initial_candidate' (one canonical candidate per task reused for both baseline and guarded episodes). The analysis plan specified paired_binary_analysis_v1 with exact McNemar test and a 95% bootstrap percentile CI computed with 10,000 resamples (bootstrap_seed = 1729). All seeds, analysis code, and CI parameters are archived.

Task synthesis and templates. Tasks were generated programmatically from a deterministic template bank (generator_id deterministic_netops_task_generator_v5, version 5.0). Each task manifest entry encodes: initial_state, expected_final_state, an explicit required_sequence (minimal safe command ordering), per-task workflow_policies (e.g., must_be_down_for_fields, minimum_active_in_groups), allowed_touched_fields, and a difficulty annotation (changed_interface_count, transient_rule_count). Templates intentionally encode operational transient constraints (e.g., make-before-break, atomic MTU changes) so validator checks capture semantic and transient safety properties rather than only syntactic validity. Representative manifest entries and their SHA-256s are published (execution/task_manifest.jsonl; example task-000001 manifest SHA-256 ee55625286dcd27c...). Inclusion rule: a task remains in the confirmatory set only if the deterministic template baseline itself parses and the gold template passes the validator unit tests (preregistered). Exclusion and replacement happen prior to model calls to maintain N = 120.

Model, orchestration, and call accounting. Declared LLM: gpt-5-mini accessed via the hosted adapter family hosted_netops_gvr_v1; the exact model configuration artifact is execution/model_configuration.json (SHA-256 a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ff4d82a). The execution contract limited maximum_model_calls = 240; because shared_initial_candidate semantics were used, the run made 120 generation calls (one canonical candidate per task) and recorded provider call traces for reproducibility (execution/provider_calls/*.json). The execution manifest records planned_episode_count = 240 and completed_episode_count = 240; model_calls_used = 120; failed_episode_count = 0. A preregistered retry policy permitted up to two automated retries for infrastructure failures; none occurred.

Generation semantics detail (shared_initial_candidate). The preregistered confirmatory semantics 'shared_initial_candidate' instructs orchestration to generate a single canonical candidate for each task (file execution/responses/task-000XXX-shared-initial.txt) and

supply that identical candidate to both scoring episodes (baseline and guarded) for deterministic comparison. This choice intentionally removes generation variance and isolates scoring differences due to scoring condition only. The shared candidate fingerprints are archived (representative: task-000001-shared-initial SHA-256 8f38c8becd7e1e36..., task-000002 SHA-256 ae967f70dfb6c..., task-000003 SHA-256 f7f4db249ecbbce...).

Deterministic validator and scoring rule. Scoring was performed with deterministic_netops_validator_v5 (validator_version 5.0). The validator pipeline (archived scoring traces under execution/scoring/*.json) implements: (1) deterministic parsing and command normalization; (2) enforcement of per-task workflow_policies (transient invariants such as minimum_active_in_groups, must_be_down_for_fields); (3) simulation of final_state reasoning (reachability and ACL equivalence checks) in a deterministic semantics engine; (4) normalized configuration canonicalization and violation enumeration. The primary scoring rubric for the confirmatory test was full_intent_constraint_validation: binary pass (score = 1) requires all required_commands present, per-task transient constraints satisfied, and validator final_state consistency with expected_final_state. Each scoring episode emits a JSON trace containing parsed_command_count, required_command_count, normalized_configuration, validation_before and validation_after states, and violation_count. Representative scorer outputs are archived (e.g., execution/scoring/task-000001-guarded.json SHA-256 0d56c1add7c5a57f...).

Analysis procedures. Primary confirmatory estimator: compute per-task baseline_i and guarded_i binary outcomes (validator pass = 1 / fail = 0) and average the paired differences across i = 1..120. Inference: (A) exact McNemar two-sided test on discordant pairs (n_10, n_01) as preregistered; (B) 95% bootstrap percentile CI on the paired difference using 10,000 resamples (bootstrap_seed = 1729). Missingness treatment: preregistered 'complete_pair_primary' rule—exclude a pair only if both episodes failed for infrastructure reasons; sensitivity analysis treats failed model calls as failures (binary=0). Contamination checks: preregistered heuristics (exact match and normalized n-gram overlap thresholds) were computed and recorded in analysis/contamination_summary.csv; artifact traces in execution/provider_calls and call_cache permit external exposure audits.

Reproducibility and artifact indexing. All artifacts required for exact reproduction are published and SHA-256 hashed in the execution manifest (execution/execution_manifest.json). Key artifact types and representative paths: (1) initial master prompt (provenance/master_prompt.txt; SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15); (2) model configuration (execution/model_configuration.json; SHA-256 a40e96e2...); (3) shared candidates (execution/responses/*-shared-initial.txt) and per-condition responses; (4) per-episode scoring traces (execution/scoring/*.json); (5) provider call traces (execution/provider_calls/*.json) and call cache

(execution/call_cache/*) to permit deterministic replay of provider interactions; (6) analysis artifacts (analysis/paired_contingency_table.csv SHA-256 9f25790c7e..., analysis/condition_summary.csv SHA-256 9c77c96f37...). The exhaustive per-file hash manifest is included in execution/execution_manifest.json so a third party can verify byte-level identity and rerun the analysis deterministically.

Pre-specified exploratory analyses and contamination plan. The preregistration included exploratory questions (seed sensitivity, error taxonomy, rank stability via Kendall tau) and a contamination plan: flagging rules for 'suspected_contamination' (exact match OR normalized 5-gram overlap ≥ 0.95) and 'high_overlap' (normalized Levenshtein ≥ 0.9). The primary confirmatory result reports inclusive outcomes; sensitivity analyses excluding or forcing suspected items to failure are recorded as separate artifacts if executed. The artifact bundle includes the outputs of the preregistered heuristics; absence of a flag is not a proof of zero pretraining exposure and is disclosed.

Why the shared_initial design produces a degenerate paired result (diagnostic). Because identical canonical candidates per pair were evaluated by a deterministic validator, any per-pair pass/fail outcome is necessarily identical across the two conditions; deterministic scoring therefore yields n_{11} or n_{00} for each pair. The archived shared candidate files and scoring traces directly document this chain: several representative pairs (e.g., task-000001, task-000002, task-000003) show identical response SHA-256s and identical scoring JSONs across baseline and guarded episodes. The confirmatory contingency table (analysis/paired_contingency_table.csv) is accordingly degenerate: $n_{11} = 120$, $n_{10} = 0$, $n_{01} = 0$, $n_{00} = 0$. The artifact records (execution/responses/*, execution/scoring/*) provide exact, auditable evidence for this causal path.

Runnable, artifact-grounded modifications for an informative comparison. Using the exact published artifacts and code, one can perform deterministic, reproducible reruns that answer independent generative reliability: (A) independent_per_condition confirmatory sampling — generate a separate canonical seed per condition per task (e.g., baseline seed_0, guarded seed_1) and rescore; (B) multi-seed confirmatory design — generate $K \geq 3$ independent seeds per condition and estimate per-condition pass probabilities with bootstrap CIs; (C) validator diversification — add a second deterministic verifier and report intersection/union pass rates to reduce validator–task coupling. These modifications require no new unpublished data and can be implemented deterministically using the archived provider call cache and task generator code paths in the bundle.

IV. RESULTS

Confirmatory counts and inferential summary (artifact grounding). All confirmatory scoring episodes completed and are traceable in the execution manifest (execution/execution_manifest.json). The archived confirmatory counts are: baseline_success_count

= 120/120; guarded_success_count = 120/120; complete_pair_count = 120. The paired contingency table is degenerate: $n_{11} = 120$, $n_{10} = 0$, $n_{01} = 0$, $n_{00} = 0$ (analysis/paired_contingency_table.csv SHA-256 9f25790c7eeafb3...). The primary estimator (mean of guarded_i - baseline_i) = 0.00. The preregistered bootstrap (10,000 resamples, seed 1729) yields a 95% percentile CI [0.00, 0.00]; exact McNemar two-sided $p = 1.0$. Analysis artifacts (condition_summary.csv SHA-256 9c77c96f37c4b6ff..., paired_difference_figure.svg SHA-256 ae5b8e0c644f8cf7...) are published in the bundle.

Representative validator traces (artifact examples). The per-episode JSON traces record parsed_command_count, required_command_count, normalized_configuration, validation_before and validation_after final_state dictionaries, and violation_count. Examples: execution/scoring/task-000001-guarded.json (SHA-256 0d56c1add7c5a57f...) shows parsed_command_count = 4, required_command_count = 4, violation_count = 0 and contains explicit before/after final_state mappings. The canonical candidate used for that pair is archived at execution/responses/task-000001-shared-initial.txt (SHA-256 8f38c8becd7e1e36...). Multiple pairs exhibit identical baseline/guarded/shared response fingerprints (e.g., task-000003 artifacts all share SHA-256 f7f4db249ecbbce...), demonstrating that identical candidates were evaluated across both conditions.

Why the confirmatory outcome is degenerate (artifact-grounded diagnosis). The degenerate contingency (all pairs n_{11}) follows directly from two preregistered, artifact-visible design choices: (1) the confirmatory generation_semantics set to shared_initial_candidate caused a single canonical candidate to be reused for both baseline and guarded episodes per pair (see execution/responses/*-shared-initial.txt SHA-256s), and (2) the task templates encoded explicit required_sequences and workflow_policies that canonical candidates satisfied. Under these conditions a deterministic validator necessarily returns identical pass/fail judgements for identical inputs, hence the paired difference must be zero. The execution and scoring artifacts (execution/responses/* and execution/scoring/*) fully document this chain of causation and are published to enable exact reruns.

Contamination and exposure. Preregistered contamination heuristics flagged no confirmatory items under the chosen thresholds (analysis/contamination_summary.csv). All provider call traces and call_cache entries are published to permit stronger external exposure checks; absence of a flag in the preregistered heuristic is not proof of zero pretraining exposure and is explicitly noted in the Disclosure.

V. DISCUSSION

Interpretation of confirmatory inference. The reported confirmatory statistics are correct for the preregistered estimand: they test whether a deterministic verifier treats identical artifacts differently under two scoring conditions. They do not, however, estimate the operational quantity of independent

LLM generative reliability (the probability that independently sampled LLM candidates satisfy verifier constraints). For deployment decisions operators need independent_per_condition pass probabilities; the archived artifacts permit reruns under those semantics without creating new task definitions.

Artifact-grounded remediation and runnable next experiments. Using the published bundle one can immediately perform informative reruns while preserving reproducibility: (A) independent_per_condition confirmatory sampling — generate separate seeds per condition per task (e.g., seed_0 baseline, seed_1 guarded), rescore with deterministic_netops_validator_v5 and compute paired contrasts; (B) multi-seed confirmatory design — draw $K \geq 3$ independent seeds per condition per task and estimate per-condition pass probabilities with bootstrap CIs; (C) validator diversification — add a second independent verifier (e.g., reachability emulator) and report intersection/union pass rates to reduce validator–task coupling. Because the generator, provider traces, and scoring harness are fully archived (execution/model_configuration.json SHA-256 a40e96e2..., execution/provider_calls/*.json, execution/scoring/*.json), these reruns are deterministic and reproducible.

Threats to validity (artifact-identified). Internal validity: the shared_initial design intentionally removed generation variance and therefore precluded testing independent generative reliability. Construct validity: template-aligned tasks and a single deterministic validator increase the chance that canonical candidates satisfy required_sequences, possibly overstating success in open distributions. External validity: a single declared model (gpt-5-mini) and synthetic tasks limit generalization to other model families, open natural language intents, and production topologies. Conclusion validity: a degenerate contingency table produces trivial but correct inferential outputs; interpreting them beyond their precise estimand would be inappropriate. All these threats are documented in the published traces.

Operational implications for NetOps practice. Our artifact-anchored diagnosis shows that (1) verifier grounding is necessary but not sufficient—experimental semantics must match the deployment question; (2) transparent preregistration plus exhaustive artifact publication materially clarifies what a given confirmatory test measures; and (3) simple rerun protocols (independent sampling, multi-seed replication, validator diversity) convert a degenerate confirmatory design into an informative operational comparison without sacrificing reproducibility.

VI. LIMITATIONS

- Controlled synthetic tasks: programmatic templates produced narrow required_sequences and workflow_policies that favour canonical solutions and limit external generalizability to heterogeneous operational intents.
- Confirmatory shared_initial semantics: the preregistered generation_semantics = 'shared_initial_candidate' supplied identical canonical candidates to both conditions

per pair, reducing independence between conditions and causing the degenerate paired outcome.

- Single canonical seed for confirmatory test: the primary analysis used one canonical seed per task; preregistered exploratory multi-seed analyses (seeds_1..4) were not included in the confirmatory inference reported here.
- Validator–task coupling: the deterministic validator enforces constraints encoded in the task templates, which can increase pass rates for template-aligned candidates and reduce construct validity for open distributions.
- Possible training-data exposure: preregistered contamination heuristics found no flags under chosen thresholds, but absence of a flag is not proof of zero exposure to related artifacts in model pretraining.
- Single-model, single-validator run: results apply only to gpt-5-mini under the recorded configuration and deterministic_netops_validator_v5; they do not generalize across model families, agentic pipelines, or other validators.

VII. CONCLUSION

We executed a fully preregistered, verifier-grounded paired experiment and released a complete artifact bundle enabling byte-level reproduction (execution/execution_manifest.json and analysis/*). Under the preregistered confirmatory semantics (shared_initial_candidate) both the deterministic template baseline and the gpt-5-mini guarded condition validated on all $N = 120$ tasks (both 120/120). Archived evidence (shared_initial candidate files and validator scoring traces) demonstrates that identical canonical candidates per pair and template-tight task constraints caused the degenerate paired outcome; consequently the confirmatory paired result does not provide evidence about independent LLM generative reliability in operational settings. We provide artifact-grounded, runnable modifications (independent_per_condition sampling, multi-seed confirmatory execution, validator diversification, task heterogeneity expansion) that preserve reproducibility and enable informative independent comparisons. All execution and analysis artifacts—including the immutable master prompt reference (provenance/master_prompt.txt; SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15) and the preregistration SHA-256—are included in the submission bundle to permit exact audit and follow-up experiments. Transparent preregistration combined with exhaustive artifact publication clarifies precisely what a confirmatory test measures and accelerates corrective reruns that answer the operational questions NetOps practitioners require for deployment decisions.

DISCLOSURE STATEMENT

Mandatory disclosure (CNSM Agentic AI Researcher track). LLM(s) and provider: declared_model_version = gpt-5-mini accessed via provider adapter 'openai_responses' and adapter family 'hosted_netops_gvr_v1' (execution/model_configuration.json SHA-256

a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd820).

Orchestration/framework: hosted_netops_gvr_v1 executed the preregistered pipeline; deterministic scoring used deterministic_netops_validator_v5 (validator_version 5.0).

Preregistration: CAND-2026-001-NetConfig-Semantic-v1 (frozen prior to execution); preregistration SHA-256 7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fdedb71a37.

Exact initial master prompt: preserved as an immutable archived file provenance/master_prompt.txt (SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba).

Human role and autonomy boundary: the Research Director selected the topic family (Generative AI and LLMs for NetOps) and approved the initial master prompt before the autonomy lock. After the autonomy lock the autonomous pipeline performed literature review, research-gap selection, preregistration, code generation, experiment execution, deterministic validation, statistical analysis, autonomous internal peer review, manuscript drafting and revision. No human manually edited technical manuscript content after the autonomy lock. Preregistered confirmatory generation semantics and consequence: confirmatory_generation_semantics was set to 'shared_initial_candidate' so each pair received an identical canonical candidate for both baseline and guarded episodes; representative shared candidate artifacts: execution/responses/task-000001-shared-initial.txt (SHA-256 8f38c8becd7e1e36...), execution/responses/task-000002-shared-initial.txt (SHA-256 ae967f70dfb6c...), execution/responses/task-000003-shared-initial.txt (SHA-256 f7f4db249ecbbce...). Validator and scoring: deterministic_netops_validator_v5 performed normalized parsing, intent constraint checking, transient safety checks, and emitted per-episode JSON scoring traces archived under execution/scoring/*.json (representative SHA-256s: execution/scoring/task-000001-guarded.json SHA-256 0d56c1add7c5a57f...). Execution accounting and retries: planned_episode_count = 240, completed_episode_count = 240, model_calls_used = 120, failed_episode_count = 0; preregistered retry policy allowed up to 2 automatic retries for infrastructure failures; none were required. Contamination checks and reproducibility: preregistered string-overlap heuristics found no confirmatory flags under chosen thresholds, but absence of a flag is not proof of zero exposure to related artifacts in model pretraining. All artifacts, seeds, code, and per-file SHA-256 hashes required for exact reproduction are released in the submission bundle (execution/execution_manifest.json contains the exhaustive manifest). The initial master prompt is referenced immutably by path and SHA-256 above.

REFERENCES

- [1] <https://doi.org/10.1145/3603269.3604866>
- [2] <https://doi.org/10.23919/cnsm62983.2024.10814407>
- [3] <https://doi.org/10.1002/nem.70029>